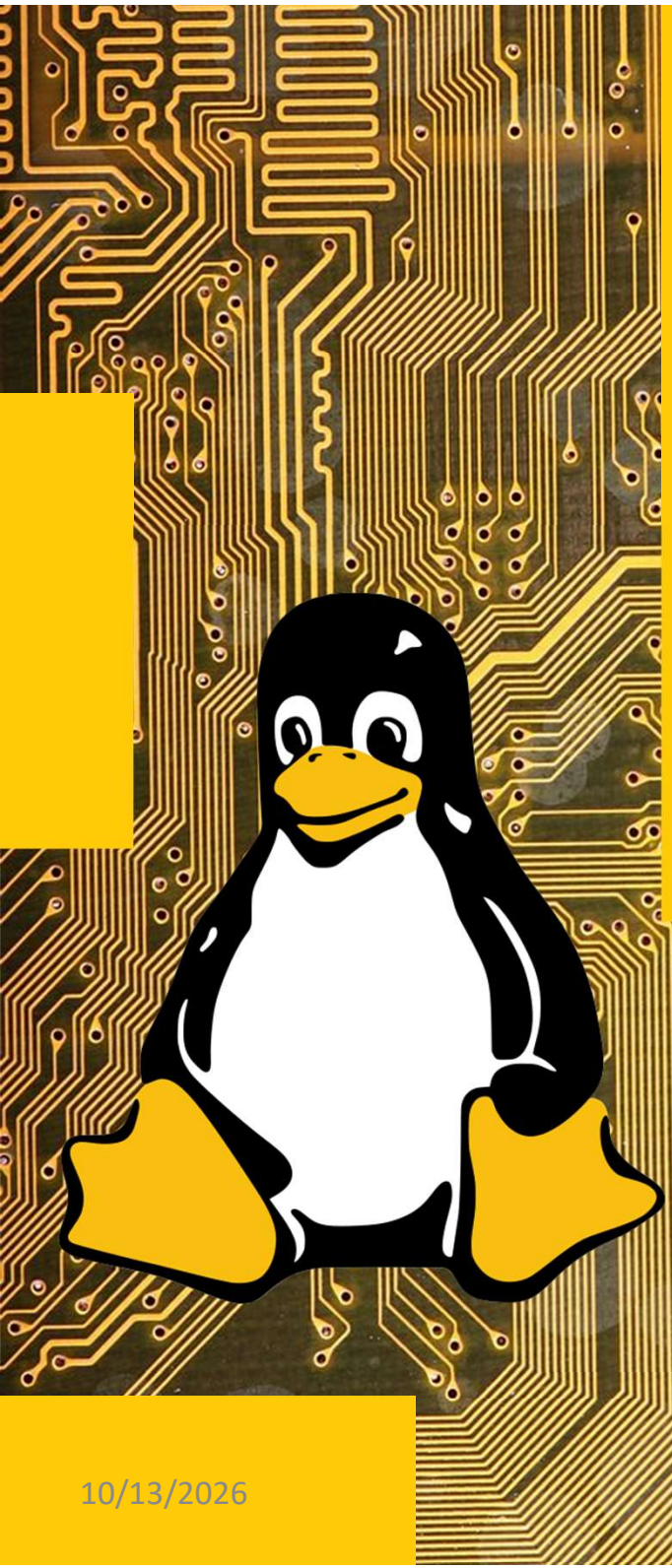


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

Synchronisation I

Examples on Paper

INSTRUCTOR: Jalolliddin Yusupov

Four Patterns, One Table

pattern	start values	who waits	who posts	in the café
limit	`stools = N`	every customer, before sitting	the same customer, when leaving	three stools at the counter
rendezvous	`doneP1 = 0` `doneP2 = 0`	each friend, for the other's flag	each friend, his own flag — first	two friends leave together
barrier	`mutex = 1`, `gate = 0` `arrived = 0` (shadow counter)	everybody, at the gate	the last to arrive opens it; each one who passes re-opens it	the tour group
shelf	`empty = N`, `full = 0` `mutex = 1`	the cook on `empty`, the waiter on `full`, both on `mutex`	the cook posts `full`, the waiter posts `empty`	the pass shelf

The trace table, in four lines

One row per `wait` or `post` · each value column holds the value **after** the step · a `wait` on 0: write the name under *sleeping* · a `post` to a sleeper: the value stays, the sleeper wakes.

Two rules that never change

Set the start value once, then only wait and post — never read the value. On the shelf: wait for a place first, then for the key.

The start value is the meaning.

Exercise: Five Customers, Three Stools

#	operation	S after	sleeping	inside
0	(start)	3	—	—
1	C1: wait(S)			
2	C2: wait(S)			
3	C3: wait(S)			
4	C4: wait(S)			
5	C1: post(S)			
6	C5: wait(S)			
7	C3: post(S)			
8	C2: post(S)			
9	C4: post(S)			
10	C5: post(S)			

Exercise · on paper

```
S = Semaphore(3)        /* three stools */

Customer C1 ... C5:
  wait(S)               /* sit down     */
  eat
  post(S)               /* free a stool */
```

(a) The ten steps happen in the order printed. Fill in **S after**, **sleeping** and **inside** (past its `wait`, not yet at its `post`) for every row.

(b) Could a fourth customer ever be seated while three are eating? Point to the row that proves it.

(c) After row 10, S is back where it started. Why **must** it be?

One row per wait or post: a sleeper changes nothing until somebody posts.

Answer: Five Customers, Three Stools

#	operation	S after	sleeping	inside
0	(start)	3	—	—
1	C1: wait(S)	2	—	C1
2	C2: wait(S)	1	—	C1, C2
3	C3: wait(S)	0	—	C1, C2, C3
4	C4: wait(S)	0	**C4**	C1, C2, C3
5	C1: post(S)	0 (flag goes to C4)	—	C2, C3, C4
6	C5: wait(S)	0	**C5**	C2, C3, C4
7	C3: post(S)	0 (flag goes to C5)	—	C2, C4, C5
8	C2: post(S)	1	—	C4, C5
9	C4: post(S)	2	—	C5
10	C5: post(S)	3	—	—

(b) Never four · (c) back to 3

inside never holds more than three names: the fourth `wait` (rows 4 and 6) finds S = 0 and sleeps. And every `wait` was matched by one `post`: flags are never made or destroyed, only moved.

You have already seen a limit

Week 5's **pool of 2**: at most two clients served at once. The **listen queue**: at most a fixed number of callers on hold.

```
$ time ./stools
C1 sits
C2 sits
C3 sits
C3 leaves
C2 leaves
C4 sits
C5 sits
C1 leaves
C4 leaves
C5 leaves
real    0m2.006s
```

A limit is a lock with more than one key.

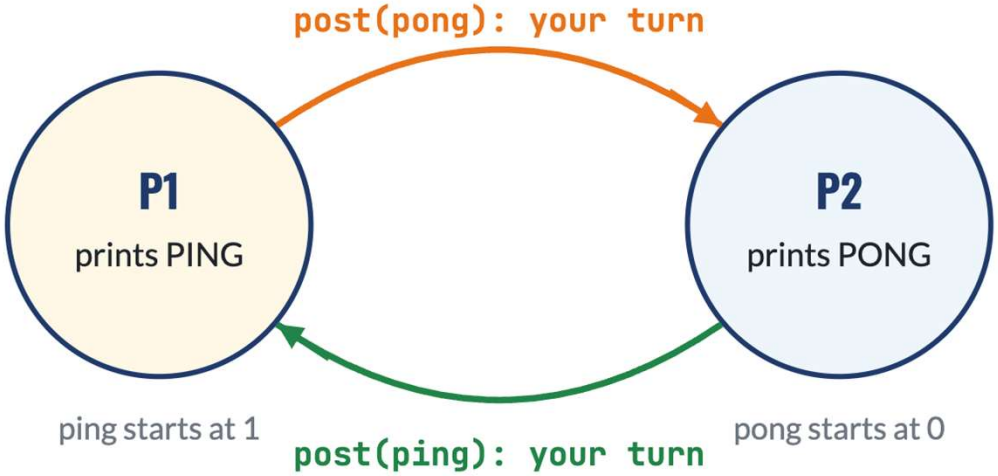
Ping-Pong for Ever

rendezvous happens once

Week 4 ended with two semaphores in shared memory, `ping` starting at 1 and `pong` at 0. Here is the whole program:

```
ping = Semaphore(1)
pong = Semaphore(0)

Process P1:                Process P2:
  loop {                     loop {
    wait(ping)                 wait(pong)
    print("PING")              print("PONG")
    post(pong)                 post(ping)
  }                          }
```



A before B, in a circle

P1's PING is the "A" that P2 waits for; P2's PONG is the "A" that P1 waits for. Each `post` hands the turn to the other.

Exercise · change only the start values

(a) `ping = 1`, `pong = 0`: what is printed?

(b) Both start at 1. What can be printed now? Can PING appear twice in a row? Three times?

(c) Both start at 0. What happens?

Hint. Count the flags. How many exist at the start, in which hole — and who can take each one?

Same code, different start values, different program.

Answer : Who Goes First?

(a) 1 and 0	`PING PONG PING PONG …` for ever: strict turns, PING first.
(b) 1 and 1	The order is **no longer forced**: each process can run ahead by one. In every pair of lines there is one PING and one PONG, either first: `PING PONG PONG PING PING PONG` is possible. Two PINGs in a row: **yes**. Three: **never**.
(c) 0 and 0	**Nothing** is printed. Both sleep in their first `wait` : a deadlock again.

Why never three PINGs?

Each PING uses a `ping` flag and only a PONG gives one back. Start with one spare flag and you can be at most one line ahead.

Reading the commands

`head -6` shows the first 6 lines; `uniq -c` counts equal lines that come one after another; `wc -l` counts what is left. `awk '$1>1'` keeps the counts above 1: **0** = never two equal lines in a row. `awk '$1>2'` keeps the counts above 2: **0** = never three in a row. The output goes to a file first: with `| head` , the writer is killed in the middle of its turn and the other process sleeps for ever.

In a cycle, exactly one semaphore starts at 1 — it says who goes first.

PING = 1, PONG = 0

```
$ ./pingpong > out.txt
$ head -6 out.txt
PING
    PONG
PING
    PONG
PING
    PONG
$ uniq -c out.txt | awk '$1>1' | wc -l
0
```

BOTH START AT 1

```
$ ./pingpong_both1 > out.txt
$ uniq -c out.txt | awk '$1>2' | wc -l
0
$ uniq -c out.txt | head -6
      1     PONG
      2 PING
      2     PONG
      2 PING
      2     PONG
      1 PING
```

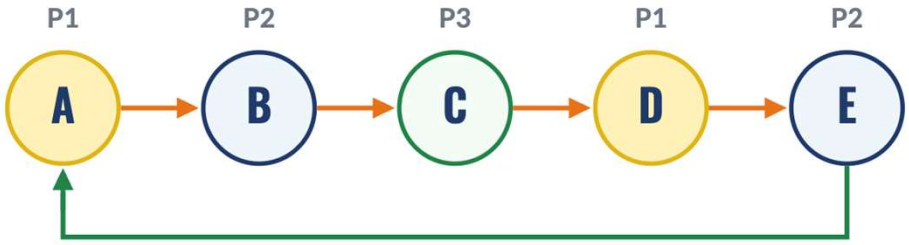
(b) Both start at 1.

Exercise: ABCDE for Ever (more arrows now)

Exercise · fill in the blanks

```
semaphores and start values: ____________________

Process P1:     Process P2:     Process P3:
  loop {          loop {          loop {
    --------        --------        --------
    print("A")      print("B")      print("C")
    --------        --------        --------
    --------        --------      }
    print("D")      print("E")
    --------        --------
  }               }
```



the cycle closes: E → A

an arrow X → Y means: X must be printed before Y

Put a `wait(…)` or a `post(…)` in the blanks (some stay empty) and choose the start values, so that the output is **ABCDEABCDEABCDE**. **for ever**, whatever the scheduler does.

In a cycle, the start values are the whole puzzle.

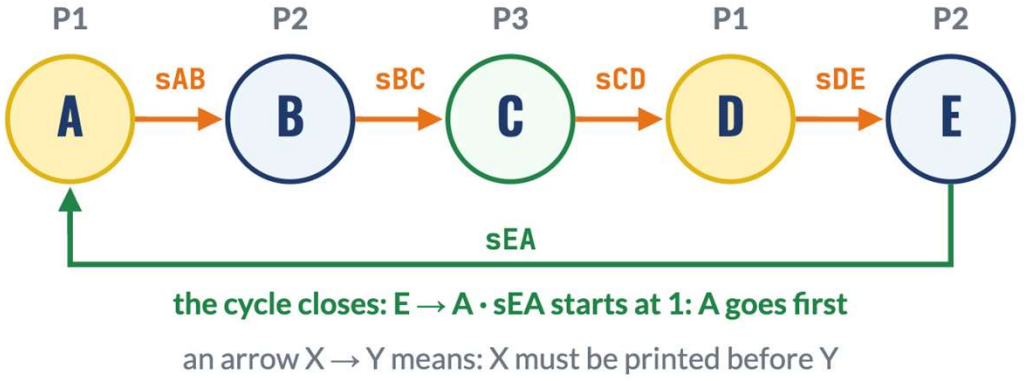
The rules

One semaphore per arrow, nothing else. Before printing, a process waits on every arrow coming **in**; after printing, it posts on every arrow going **out**. State every start value.

Hint

Count the arrows: that is how many semaphores you need. In a cycle, somebody has to be allowed to start — how many flags exist at the beginning?

Exercise: ABCDE for Ever



Why nothing else can be printed

Each letter's only flag is posted by the letter before it, so the order is forced. Exactly one flag exists at the start (in sEA): it says that A goes first.

```
sAB = sBC = sCD = sDE = Semaphore(0)
sEA = Semaphore(1)                  /* A goes first */

Process P1:      Process P2:      Process P3:
  loop {           loop {           loop {
    wait(sEA)        wait(sAB)        wait(sBC)
    print("A")       print("B")       print("C")
    post(sAB)        post(sBC)        post(sCD)
    wait(sCD)        wait(sDE)      }
    print("D")       print("E")
    post(sDE)        post(sEA)
  }                }
```

A→B and D→E	**may share.** Both go from P1 to P2; P2 needs them in this order, and the second is posted only after the first was taken. 4 semaphores.
A→B and B→C	**never.** Different receivers: P3 can take the flag meant for P2 and print A C D — then everybody sleeps.
C→D and E→A	**prove it first.** Same receiver (P1), different senders: a flag does not say who posted it. Here the order saves you-that needs a proof.

A semaphore per arrow always works; fewer is an optimisation you must prove.

Exercise: Every Possible Output

```
mutex = Semaphore(1)
s = Semaphore(0)

Process P1:      Process P2:      Process P3:
  wait(mutex)      wait(mutex)      wait(s)
  print("A")       print("C")       print("D")
  print("B")       post(mutex)
  post(mutex)      post(s)
```

Find what cannot happen first; what is left is what can.

Exercise · list every output

(a) Four letters can be printed in 24 orders. Which of them can this program print? List **all** of them.

(b) For two orders that are **impossible**, give the reason in one line each.

YOUR ANSWER

output	possible?	why (one line)

Hint

Ask two questions. Which lines can never run at the same time as which? Which letter has to wait for a flag — and who posts it, and when?

The rules of the game

The scheduler may run the three processes in any order and switch after any line. Each `print` prints one letter. The program always finishes.

Answer: Exactly Four Outputs

```
mutex = Semaphore(1)
s     = Semaphore(0)

P1: wait(mutex); print A; print B; post(mutex)
P2: wait(mutex); print C; post(mutex); post(s)
P3: wait(s); print D
```

output	how it happens
`A B C D`	P1 takes the key first and prints A B; then P2 prints C and posts `s`, which lets P3 print D.
`C A B D`	P2 takes the key first: C. Then P1 prints A B before P3 gets its turn.
`C A D B`	P2 first; P3's D slips in **between** A and B — P3 never takes the key, so the key cannot stop it.
`C D A B`	P2 first; P3 prints D before P1 even starts.

4 of the 24 orders of A, B, C, D are possible — checked by trying every interleaving.

Why the other 20 are impossible

Rule 1 — nothing between A and B except D. P1 holds the key from A to B, so C (also under the key) cannot come between them: `A C B D` is impossible.

Rule 2 — D after C. D needs the flag in `s`, and P2 posts it only after printing C: `D C A B`, `A B D C` and every order with D before C are impossible.

The method

Two kinds of semaphore, two kinds of rule: the **key** (start 1) keeps lines apart; the **signal** (start 0) puts one line after another. Write both rules down, then list what survives.

A key keeps lines apart; a signal puts them in order.

The trap

D **can** fall between A and B (`C A D B`): P3 never takes the key, so the key cannot keep it out. A lock keeps out only those who also use it.

Exercise: Swap Two Lines

THE COOK'S TWO WAITS ARE SWAPPED

```
Cook:
  loop {
    cook a plate
    wait(mutex)
    wait(empty)
    put the plate
    post(mutex)
    post(full)
  }
```

```
Waiter:
  loop {
    wait(full)
    wait(mutex)
    take a plate
    post(mutex)
    post(empty)
    serve it
  }
```

The shelf at the start

N = 1, and it already holds a plate: `empty = 0`, `full = 1`, `mutex = 1`.

In pairs, five minutes

The classic-mistakes table names this bug. Prove it: find the order of steps, then compare with your neighbour.

Exercise · stop the café for ever

(a) Find an order of steps after which **nobody can ever move again**. Write it as a trace table:

step	who	operation	empty	full	mutex	sleeping
0		(start)	0	1	1	—
1						
2						
3						
4						
5						

(b) One sentence: what did the cook do wrong?

Answer: The Deadlocked Shelf

step	who	operation	empty	full	mutex	sleeping
0		(start)	0	1	1	—
1	Cook	wait(mutex)	0	1	0	—
2	Cook	wait(empty)	0	1	0	Cook (holding the key)
3	Waiter	wait(full)	0	0	0	Cook
4	Waiter	wait(mutex)	0	0	0	Cook, Waiter — everyone: nobody can move

Who waits for whom

The cook waits for `empty`: only the waiter posts it, after taking a plate **under the key**. The waiter waits for the **key**: only the cook can give it back. A circle of waiting.

(b) What the cook did wrong

He went to sleep **holding the key**. Had the waiter moved first, the café would have survived — that is why such bugs pass tests.

The OS did nothing wrong

Both asked to sleep, and the OS let them: `TIME 00:00:00`, no CPU used.

```
$ timeout 5 ./shelf_deadlock 1000; echo "exit code $?"
exit code 124
$ ./shelf_deadlock 1000 &
$ sleep 3; ps -o pid,stat,time,comm
  PID STAT     TIME COMMAND
    1 Ss   00:00:00 bash
  175 S    00:00:00 shelf_deadlock
  177 S    00:00:00 shelf_deadlock
  178 S    00:00:00 shelf_deadlock
  181 R    00:00:00 ps
```

Never go to sleep while holding the key.

Exercise: Three Broken Shelves

Every café: **N = 4**, one cook, one waiter, `empty = Semaphore(N)`, `full = Semaphore(0)`, `mutex = Semaphore(1)` — except where shown.

THE COOK — X, Y AND Z

```
Cook:
loop {
  wait(empty)
  wait(mutex)
  put plate k
  print("cooked", k)
  post(mutex)
  post(full)
}
```

SHELF X — WAITER

```
Waiter:
loop {
  wait(full)
  wait(mutex)
  take plate k
  print("served", k)
  post(mutex)
  /* nothing here */
}
```

SHELF Y — WAITER

```
Waiter:
loop {
  wait(empty)
  wait(mutex)
  take plate k
  print("served", k)
  post(mutex)
  post(empty)
}
```

SHELF Z — START VALUES

```
empty = Semaphore(N)
full  = Semaphore(0)
mutex = Semaphore(0)

/* the cook and the
   waiter are both
   correct */
```

Each shelf has one wrong line. What do you see on the screen?

For each one: say **what is printed**, **who ends up asleep** (and on which semaphore), and write the **one-line fix**.

Read it like the trace table

Each cook step uses one free place (empty) and makes one plate (full).
Each waiter step should do the opposite. Who gives back what — and who can never start?

One wrong line is enough: read every wait and every post.

Answer: Three Broken Shelves

Each semaphore has one job: count places, count plates, or guard the shelf.

bug	what you see	fix
X — the waiter never posts `empty`	`cooked 1` … `cooked 4` and `served 1` … `served 4`, then **silence**. The cook used up the 4 free places and nobody gave one back: he sleeps on `empty`, the waiter on `full`.	`post(empty)` after taking the plate
Y — the waiter waits on `empty`, not `full`	**served** lines for plates that were never cooked (an empty place, or a plate served twice), and real plates lost. The waiter's wait and post on `empty` cancel out, so after 4 plates the cook still runs out of places: then both sleep on `empty`.	`wait(full)`
Z — `mutex` starts at 0	**Nothing at all.** The cook passes `wait(empty)` and sleeps on the key; the waiter sleeps on `full`. Asleep from the first second.	`mutex = Semaphore(1)`

One semaphore, one job

`empty` counts places, `full` counts plates, `mutex` is the key. Each bug gives one of them the wrong job — or none.

Same disease as the servers of Week 5

Each program works for a moment, so a quick test can pass. Only reading every `wait` and `post` — or a trace table — finds the line.

What the OS Did for You This Week

the OS …	so that …
put a waiting process to **sleep**	a cook waiting for a free place costs no CPU (state `S` in `ps`)
woke **exactly one** sleeper on each `post`	one free stool, one customer — never two
made `wait` and `post` **atomic**	two `wait`s can never both take the last flag
kept the semaphores in **shared memory**	the cook and the waiter see one counter, not two copies (made before `fork`)
let a deadlocked café sleep	nothing to run, nothing burning; `timeout` had to end it (124)
did **nothing** when a holder died	the flag is gone for ever; everyone else who needs it sleeps
runs its own bounded buffers with the same idea	the **pipe** (the writer sleeps when it is full) and the **listen queue** (callers on hold)

The OS keeps the counter honest; using it in the right order is your job.

Patterns in the Wild

who	what	the job
nginx	`accept_mutex` : a lock in front of `accept` , so the worker processes of its pool take new callers in turn — Week 5's design 3, made orderly	a lock
PostgreSQL	one semaphore for every server process, in its shared memory (`sem_init(…, 1, …)` , our four calls); a process that must wait for a busy lock sleeps on it	a signal
PostgreSQL `max_connections`	at most `max_connections` clients (100 by default); when every slot is taken, the next one is refused at once (*"sorry, too many clients already"*) — a trywait, not a wait	a limit
Python	`multiprocessing.Semaphore(3)` : the limit, for processes, in one line	a limit
the pipe	the OS's own shelf: 64 KiB of bytes, the writer sleeps when it is full, the reader when it is empty	a bounded buffer

No new API here

Every line uses a job you can now trace on paper. When you read a real server's code, look for the start values first.

Real servers are built from the same five jobs.



Questions?